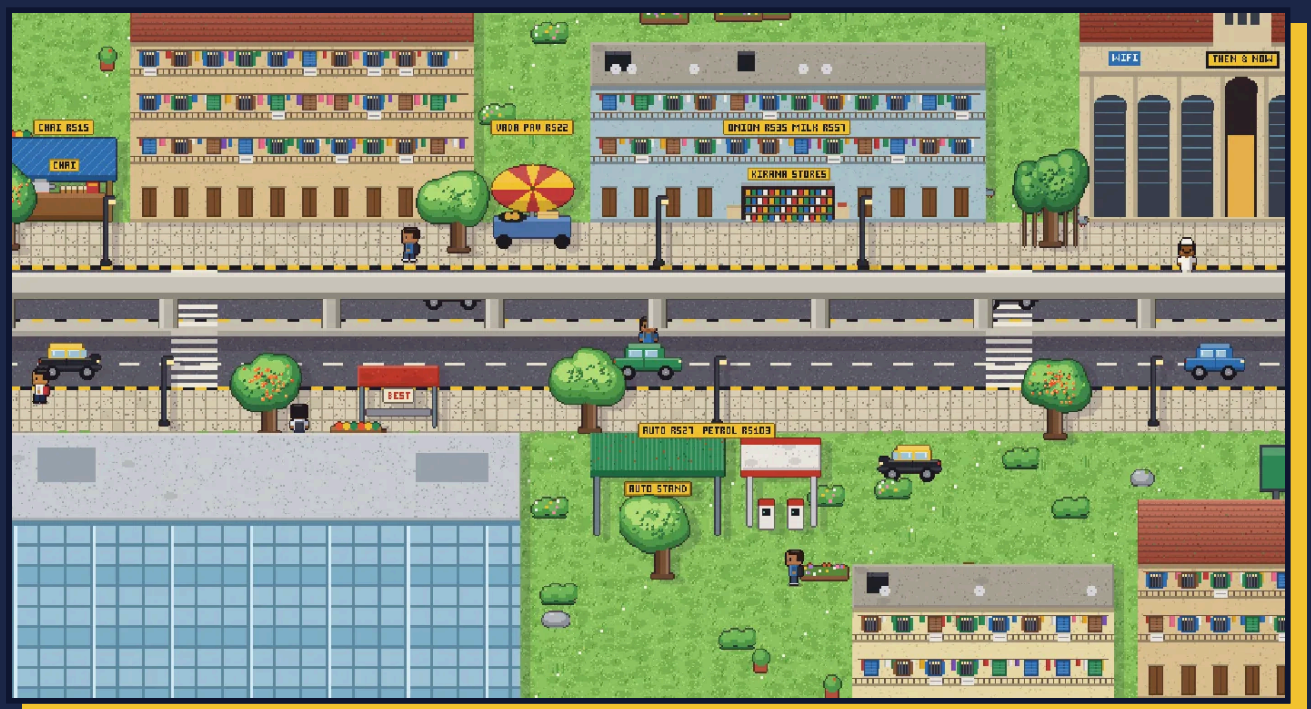
GAME GUIDE AND PRODUCT DOCUMENT

THEN & NOW
MUMBAI

What the game is, how to play it, and how it is made, including every programming language it uses, in plain words.



PLAY
then-and-now-mumbai.vercel.app

PLATFORMS
Web, Android, Windows, Mac, iPhone (home screen)

VERSION
October 2026

01 THE IDEA

A game that answers a question everyone in India asks: "How much did this cost back then?"

The problem

People love comparing prices. "Vada pav was two rupees!" "A local train ticket was three!" But nobody kept a proper record of street prices over the years. Official numbers track inflation in general, not your cutting chai. So these conversations run on memory, and memory is often wrong.

The answer

Then & Now turns 30 years of real street prices into a city you can walk around. Every shop is a little piece of data. Move the **Time Machine** slider from 1995 to 2026 and the prices, the vehicles and the buildings all change in front of you.

IN ONE SENTENCE

A pixel-art city where you walk into shops and see what street prices really were in any year from 1995 to 2026, with every number either sourced or honestly marked as a guess.

Who it is for

- **Curious people** who want to settle "what did it cost back then" arguments.
- **Students and teachers** learning about inflation, using things they know (chai, onions, petrol) instead of charts.
- **Friends and families** who want to explore together: up to 8 people can walk the same street.
- **Data lovers**, who get proper charts and the source behind every number.

Goals

1. Make price history **fun to explore**, not a spreadsheet.
2. Be **honest**: every price shows how sure we are about it.
3. Work **everywhere for free**: phone, computer, online and offline.

02 How to play

Five minutes from opening it to knowing what chai cost in 1995.

1. **Open the game.** Go to the website and press *Play*, or open the app on your phone or computer.
2. **Pick a city and a look.** Mumbai or Delhi. Choose your character's shirt colour and step onto the street.
3. **Walk around.** Arrow keys or W A S D on a computer, the joystick on a phone. Scroll or pinch to zoom in and out, like a drone.
4. **Go into a shop.** Walk up to a door and press *E* (or tap the button). The shopkeeper tells you today's price and the price in the year you picked.
5. **Use the Time Machine.** Drag the slider at the bottom to any year. Watch the city change: old Padmini taxis come back, the Metro disappears, prices drop.
6. **Talk to people.** Walk up to the dabbawala, the Koli fisherwoman or the old dada on the bench. They have stories about what changed.
7. **Bring friends.** Press *Start a room*, send the link, and walk together. Press *T* to chat.



Inside the chai shop: the shopkeeper quotes then and now.



The railway station, where local trains pull in and leave.

03 WHAT'S IN IT

Every feature, what it does, and its status.

FEATURE	WHAT IT DOES	STATUS
The city	A drone view of a Mumbai or Delhi neighbourhood: roads, chawls, sea front, trees, people walking, live traffic	DONE
Walk-in shops	Chai shop, kirana and railway station you can step inside, each with a shopkeeper	DONE
Time Machine	Slide from 1995 to 2026. Prices, vehicles and buildings all change.	DONE
Zoom	Pull back to see the whole area, or zoom right in	DONE
Safe traffic	Cars and buses stop and honk for you instead of driving through you	DONE
Two cities	Mumbai and Delhi, each with its own map, people, vehicles and price data	DONE
Analyst Mode	Charts of every price, with sources and how sure we are	DONE
Survive the month	A budget game: get through four weeks of street spending in any year	DONE
Price postcards	Make a picture comparing two years, to share on WhatsApp or anywhere	DONE
Multiplayer	Up to 8 friends in one street, with chat and a shared Time Machine	DONE
The bazaar	Ask about any price or report what you paid. A reviewer checks everything first.	DONE
Works offline	After the first visit (or straight away in the apps), the game runs without internet	DONE
Apps	Download for Android, Windows and Mac. Add to home screen on iPhone.	DONE

The two cities

Mumbai: the sea front, chawls with washing on the balconies, the chai tapri, the vada pav cart, the kirana, a local train station, BEST double-deckers, kaali-peeli taxis, and the Metro from 2014.

Prices: chai, vada pav, local train, auto, petrol, onion, milk.

Delhi: the Yamuna ghats, Old Delhi havelis, a city gate, a gurudwara, Blueline buses, green CNG autos, cycle rickshaws, and a Metro that opens in December 2002.

Prices: chai, samosa, Metro, auto, petrol, onion, milk.

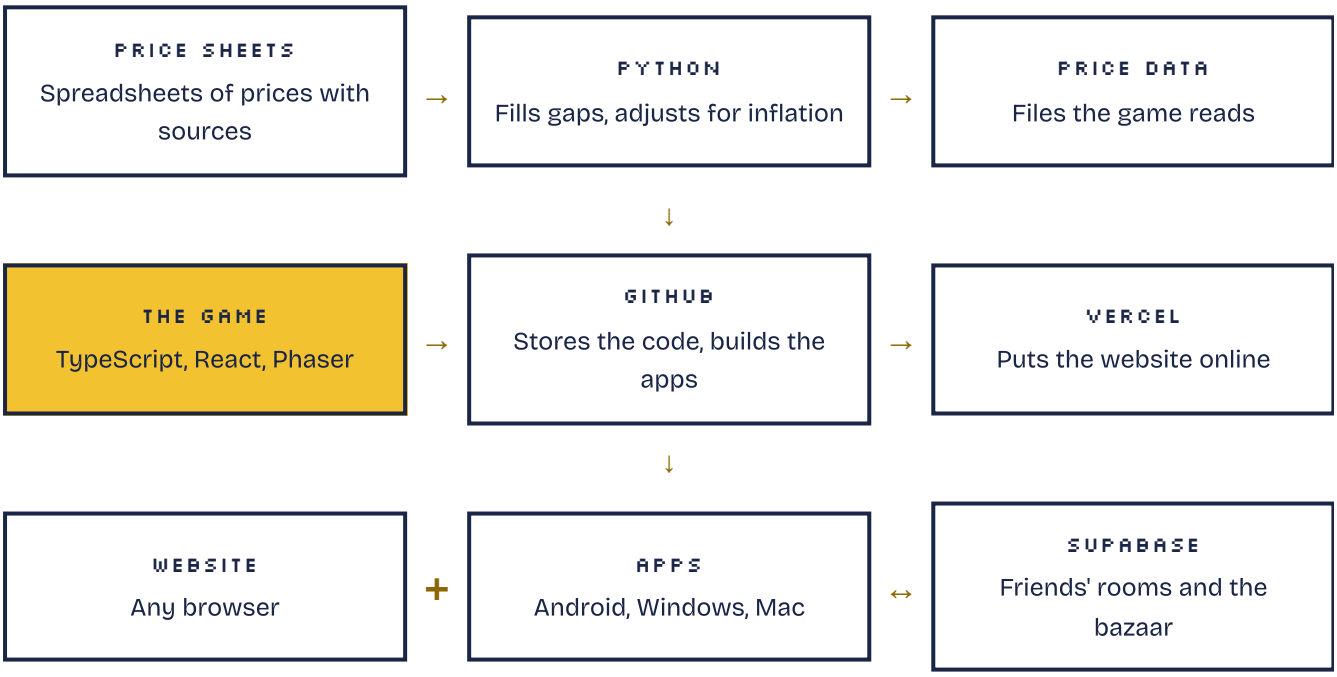
The honesty rule

Old street prices were never officially recorded, so every number carries a label:

LABEL	WHAT IT MEANS
documented	From an official notice, like a fare chart
reported	A dated newspaper report gives the number
estimated	Our best guess. Still needs a source. Shown as a hollow dot in charts.
interpolated	A year between two known prices, filled in smoothly. The shopkeeper says so.

04 HOW IT IS MADE

The whole system on one page.



THE BIG IDEA

The game is just a set of files: pages, code and price data. They run entirely on the player's phone or computer. There is no server of our own to pay for or look after. The only online parts are playing with friends and the bazaar, and Supabase handles those.

Three things that make it special

- **No picture files.** Every person, taxi, bus, tree and building is drawn by code, pixel by pixel. That is how a building can look older in 1995, or a Delhi auto can turn green when CNG arrives, just by changing a line of code.

- **A grid city.** Each city is a map of 104 × 72 squares (called tiles), each 16 pixels wide. Every square is road, pavement, grass, water, track or building.
- **Living traffic.** Vehicles drive in lanes, keep their distance, and brake and honk if you stand in the road.

The tools

TOOL	WHAT IT IS, SIMPLY	ITS JOB IN THIS GAME
Next.js	A kit for building websites	Makes all the pages and turns them into fast, simple files
React	A kit for buttons and screens	Menus, panels, charts, the Time Machine slider
Phaser	A game engine	Draws the city, moves the camera, handles walking and traffic
Supabase	An online database and live-chat service	Friends' rooms and the bazaar
Vercel	A website host	Keeps the site online and updates it on every change
GitHub	A home for code	Stores every version, and builds the apps on its own computers
Electron	Turns a website into a computer app	The Windows and Mac apps
Capacitor	Turns a website into a phone app	The Android app
Playwright	A robot that uses a browser	Tests the game by playing it and taking screenshots

05 THE LANGUAGES, ONE BY ONE

A programming language is a way of writing instructions a computer can follow. This game uses nine. For each one: what it is, what it does here, and a real piece of the game's code.

TypeScript

ABOUT 80% OF THE PROJECT

Like writing a recipe, with a label on every ingredient so you can't mix up sugar and salt.

WHAT IT IS JavaScript, the language every web browser understands, with extra checks. Every value gets a type ("this is a year", "this is a list of prices"), and mistakes show up before anyone plays.

WHAT IT DOES Almost everything: the game, the pages, the shops, multiplayer, the bazaar, postcards and the Survive game. About 9,200 lines.

WHERE `src/`, in files ending `.ts` and `.tsx` (TSX is TypeScript that also describes screens).

```
export const available = (item: Item, year: number) =>
  !item.available_from || year >= item.available_from;
```

IN WORDS "Is this item available in this year? Yes if it has no start date, or if the year is on or after its start date." This is how the Delhi Metro stays closed before 2002.

CSS

ABOUT 8%

Like the paint, fonts and furniture layout of a house.

WHAT IT IS A language that describes how things look: colours, sizes, spacing, fonts, and what happens when you hover over a button.

WHAT IT DOES The look of the website: the navy and taxi-yellow colours, the pixel font, the landing page, the shop panels.

WHERE `src/app/globals.css`

```
.ld-download {
  background: var(--taxi); color: var(--ink);
  border: 3px solid var(--ink); padding: 14px 24px;
  box-shadow: 6px 6px 0 var(--best);
}
```

IN WORDS "The Download button is taxi yellow with dark text, a thick border and a solid shadow." That is the chunky pixel-style button on the landing page.

Python

SMALL, BUT IMPORTANT

Like a calculator that does a whole spreadsheet's worth of work in a second.

WHAT IT IS — A popular, easy-to-read language, often used for data and maths.

WHAT IT DOES — Turns the price spreadsheets into the data the game reads: fills in missing years, adjusts for inflation, and works out minutes of work. It also draws the app icons.

WHERE — `scripts/build_data.py` and `scripts/make_icons.py`

```
out[y] = math.exp(math.log(anchors[lo]) * (1 - t) + math.log(anchors[hi]) * t)
```

IN WORDS "For a year between two known prices, find the price on a smooth curve between them." If vada pav was ₹2 in 1995 and ₹5 in 2005, this works out 2000 as about ₹3, growing at a steady rate rather than by the same amount each year.

SQL

THE DATABASE

Like the rules a librarian follows: who may borrow which book, and who may only look.

WHAT IT IS — The language for talking to databases: making tables, storing rows, asking questions of the data, and setting rules.

WHAT IT DOES — Sets up the bazaar in Supabase: the tables for questions and price reports, who is allowed to see what, the reviewer's checks, the limits on spam, and the nightly cleanup at 03:17.

WHERE — `supabase/migrations/`

```
create policy "anyone can read approved reports" on public.price_reports
  for select to anon, authenticated using (status = 'approved');
```

IN WORDS "Anyone can read a price report, but only if it has been approved." Unchecked reports stay hidden. The database enforces this itself, so no one can get around it.

JavaScript

SMALL

TypeScript's older sibling: the same language, without the labels.

WHAT IT IS	The language every web browser runs. TypeScript turns into JavaScript before it runs.
WHAT IT DOES	Two small jobs: the **service worker**, which saves the game so it opens without internet, and the **desktop app** that opens the game in its own window on Windows and Mac.
WHERE	`public/sw.js` and `desktop/main.js`

```
fetch(req)
  .then((res) => putInCache(req, res))
  .catch(async () => (await caches.match(req)) || (await caches.match("/play/")))
```

IN WORDS "Try to get the page from the internet and keep a copy. If there's no internet, use the saved copy, or at least open the game."

YAML

INSTRUCTIONS FOR GITHUB

Like a to-do list you hand to a helper.

WHAT IT IS	A simple format for writing settings and step-by-step instructions as indented lists.
WHAT IT DOES	Tells GitHub how to build the apps: borrow a Mac, a Windows PC and a Linux machine, build each app, check it, and publish the download files.
WHERE	`.github/workflows/apps.yml`

```
mac:
  runs-on: macos-latest
  steps:
    - name: Build the Mac app
      run: npx electron-builder --mac
```

IN WORDS "For the Mac app: use GitHub's latest Mac computer, then run the command that builds the Mac app."

Java

ONE FILE

Android's home language, used here for one small fix.

WHAT IT IS	A long-standing language that Android apps are traditionally written in.
WHAT IT DOES	Inside the Android app, it makes sure each page (the game, the bazaar, the charts) opens its own file instead of always the home page.
WHERE	`mobile/android/app/src/main/java/…/MainActivity.java`

```
InputStream page = getAssets().open("public" + dir + "index.html");
return new WebResourceResponse("text/html", "UTF-8", page);
```

IN WORDS "When the app asks for a page like /play/, open that page's file from inside the app and show it."

Gradle

ANDROID SETTINGS

The settings form for building an Android app.

WHAT IT IS	Android's build tool, with its own short settings language.
WHAT IT DOES	Sets the app's version number (so new versions install over old ones) and signs the app with its key.
WHERE	`mobile/android/app/build.gradle`

CSV and JSON

DATA, NOT CODE

CSV is a spreadsheet saved as text. JSON is the same information arranged for a computer to read.

WHAT THEY ARE	Formats for storing information. They don't give instructions; they hold facts.
WHAT THEY DO	CSV holds every price, year, source and note, and anyone can open it in Excel or Google Sheets. JSON is what the Python script writes for the game to read.
WHERE	`data/raw/*.csv` and `public/data/*.json`

```
item,year,price,confidence,source_id,note
vadapav,1995,2,estimated,,Outlook Traveller traces the 1970s Dadar original to 25 paise.
```

IN WORDS "Vada pav, 1995, ₹2. This is an estimate, with a note about where the clue came from."

Why so many languages?

Each one is the natural choice for its job, like using a pen for writing and a brush for painting. TypeScript does nearly everything. The others each do one small job they are best at: SQL for the database, Python for number crunching, YAML for build instructions, Java for one Android fix.

06 HOW PLAYERS GET IT

One game, five ways in, all free.

DEVICE	HOW	FIRST TIME
Any browser	Open the website and press Play	Nothing to install
Android phone	Download for Android: `.apk` file	Allow "install unknown apps" once
Windows PC	Download for Windows: `.exe` installer	"More info", then "Run anyway"
Mac	Download for Mac: `.dmg` file	System Settings, Privacy & Security, "Open Anyway"
iPhone / iPad	Safari, Share, Add to Home Screen	Nothing else

WHY THE FIRST-TIME WARNINGS

Apple, Microsoft and Google charge app makers to vouch for them. We haven't paid, so each system shows one warning the first time. The game is the same either way, and the website walks people through it.

How a change goes live

1. A change to the game is saved and sent to GitHub.
2. Vercel notices, rebuilds the website and puts it live, in about a minute.
3. GitHub builds fresh Android, Windows and Mac apps on its own computers, in about five minutes, and the download buttons point at them automatically.

07 WHAT'S NEXT

Ideas for future versions, roughly in order.

- **More sources.** Turn the estimated prices (hollow dots) into sourced ones from old newspapers, fare charts and receipts.

- **AI-drafted answers in the bazaar.** An assistant drafts a cited answer, and the reviewer only checks it.
- **More cities.** Kolkata, Chennai, Bengaluru, each with its own map and prices.
- **Store listings.** Google Play ($25 once) and the Apple App Store ($99 a year) for one-tap installs without warnings.
- **A custom domain**, such as thenandnow.in.

Limits to know about

- Many old prices are still estimates, and the game says so openly.
- The free Supabase plan pauses after a week with no visitors. One click wakes it.
- The Android app needs a permanent signing key saved in GitHub so updates install smoothly (see the handbook).

How we'll know it's working

- People play more than one year and visit more than one shop per visit.
- Postcards get shared.
- Players send in sourced prices through the bazaar, and the number of hollow dots goes down.

Then & Now, game guide and product document, October 2026. The detailed technical handbook is `docs/HANDBOOK.md` in the code repository.